

# กลุ่ม [8] — [Structs & Methods]

*Template สไลด์นำเสนอ — แทนที่ข้อความในวงเล็บ [ ] ด้วยเนื้อหาของกลุ่ม*

1. [นายพงศ์ชยุตม์ หัวใจเพชร] — Concept + Short Code Illustration
2. [นายพชรพล อาม่วง] — Detailed Code + Live Demo
3. [นายพิชัยพร พลบเนียม] — Rust vs Other Language + PPL Analysis
4. [นายภัทรดนัย คนชม] — Exercises + Common Mistakes + Challenge

# Agenda — โครงสร้างการนำเสนอ (รวม ~20 นาที)

แต่ละคนนำเสนอประมาณ 5 นาที ต่อเนื่องกันตามลำดับ

1

Member 1

**Concept + Short Code  
Illustration**

แนวคิดหลักของหัวข้อ พร้อมโค้ดตัวอย่างสั้น ๆ

~5 นาที

2

Member 2

**Detailed Code + Live Demo**

อธิบายโค้ดเชิงลึกและสาธิตการทำงานจริง

~5 นาที

3

Member 3

**Rust vs Other Language +  
PPL Analysis**

เปรียบเทียบภาษา และวิเคราะห์เชิง  
Programming Language

~5 นาที

4

Member 4

**Exercises + Common  
Mistakes + Challenge**

ข้อผิดพลาดที่พบบ่อย และโจทย์ท้าทายผู้ฟัง

~5 นาที

# [Structs & Methods] คืออะไร?

- Struct ใช้สร้างชนิดข้อมูลที่รวมข้อมูลที่เกี่ยวข้องกันไว้ด้วยกัน โดย Fields คือข้อมูลภายใน Struct และสามารถใช่เพื่อทำ Data Modeling ได้
- **เหตุผลที่ Rust ออกแบบแนวคิดนี้** : เพื่อจัดระเบียบข้อมูลและพฤติกรรมที่เกี่ยวข้องกัน โดยใช้ Methods และ Associated Functions ผ่าน impl ทำให้มีความชัดเจนในการเขียนโปรแกรม
- **ช่วยแก้ไข** : ช่วยลดการจัดการข้อมูลที่กระจัดกระจาย และทำให้ข้อมูลกับพฤติกรรมของสิ่งนั้นอยู่ในโครงสร้างเดียวกัน
- **จำ** : Struct = โครงสร้างข้อมูล | Fields = ข้อมูล | Methods = พฤติกรรมของ Instance | Associated Functions = ฟังก์ชันของ Type | Data Modeling = การจำลองสิ่งต่าง ๆ ด้วยข้อมูลและพฤติกรรม  
Self = รับ Instance เป็นเจ้าของ | &self ยืม Instance มาใช้งาน/อ่านข้อมูลโดยไม่แก้ไข | &mut self ยืม Instance แบบแก้ไขได้

ตัวอย่าง

```
struct Student {
    name: String,
    age: i32,
}
```

## 1 Struct 2 Fields

```
impl Student {
    fn new(name: String) -> Student {
        Student {
            name,
            age: 18,
        }
    }
}
```

## 3 Associated Function 4 Methods

```
fn introduce(&self) {
    println!("Hi, I'm {}. I'm {} years old.", self.name, self.age);
}

fn main() {
    let student = Student::new(String::from("John"));
    student.introduce();
}
```

# [Structs and Fields] แนวคิด?

Struct คือการสร้างรูปแบบข้อมูลขึ้นมาเอง เพื่อรวบรวมข้อมูลหลายอย่างที่เกี่ยวข้องกันไว้ด้วยกัน

เช่น เราต้องการเก็บข้อมูลหนังสือ 1 เล่ม

เรากำหนดว่า Book ต้องมีข้อมูล 4 อย่าง

- title → ชื่อหนังสือ
- author → ผู้เขียน
- pages → จำนวนหน้า
- is\_available → หนังสือว่างหรือไม่

สิ่งเหล่านี้เรียกว่า **Fields**

ตัวอย่าง

## 1 Struct

```
struct Book {  
    title: String,  
    author: String,  
    pages: u32,  
    is_available: bool,  
}
```

## 2 Fields

ตัวอย่าง

```
let my_book = Book {  
    title: String::from("The Rust Programming Language"),  
    author: String::from("Pattaradanai Konchom"),  
    pages: 552,  
    is_available: true,  
};
```

## [Modifying Structs] แนวคิด?

การแก้ไขข้อมูลที่อยู่ใน Struct

โดยปกติ Rust จะไม่ให้เราแก้ค่าตัวแปรที่ประกาศแบบธรรมดา

ตัวอย่าง

```
let user1 = User {  
    username: String::from("rust_coder"),  
    active: true,  
    sign_in_count: 1,  
};
```

```
user1  
  
username = rust_coder  
active = true  
sign_in_count = 1
```

```
user1.active = false;
```



Rust จะไม่อนุญาต เพราะ user1 ไม่ได้ประกาศเป็น mut จึงไม่สามารถแก้ไขค่าใน Struct ได้

# แล้วต้องทำอะไร?

ต้องใส่ `mut` ตอนประกาศตัวแปร (`mut` ย่อมาจาก `mutable`)

ตัวแปรนี้สามารถถูกแก้ไขค่าได้



ตัวอย่าง

```
let mut user1 = User {
    username: String::from("rust_coder"),
    active: true,
    sign_in_count: 1,
};
```

ค่าที่ได้

```
user1
username = rust_coder
active = true
sign_in_count = 1
```

ค่าที่ได้

```
user1.active = false;
```



```
user1
username = rust_coder
active = false
sign_in_count = 1
```

# [Tuple Structs] แนวคิด?

**Tuple Struct** คือ Struct ที่มีชนิดข้อมูล แต่ไม่มีชื่อของ Fields  
เหมาะสำหรับการรวมข้อมูลที่เกี่ยวข้องกันไว้ด้วยกัน โดยเข้าถึงข้อมูลผ่าน  
ตำแหน่ง

ตัวอย่าง

```
struct Color(i32, i32, i32);

fn main() {

    let black = Color(1, 2, 3);

    println!(

        "Black RGB: ({}, {}, {})",
        black.0, black.1, black.2

    );

}
```

ผลลัพธ์

```
black (1, 2, 3)
  ↓ ↓ ↓
  .0 .1 .2
```

# [Unit-Like Structs] แนวคิด?

**Unit-Like Struct** คือ Struct ที่ไม่มี Fields และไม่มีข้อมูลภายใน ใช้สำหรับสร้าง Type ที่ไม่จำเป็นต้องเก็บข้อมูล

ตัวอย่าง

```
struct AlwaysEqual;  
  
fn main() {  
    let _subject = AlwaysEqual;  
}
```

→ สร้าง Struct ชื่อ AlwaysEqual  
ที่ **ไม่มี Field**

→ สร้างตัวแปร \_subject  
จาก Type AlwaysEqual

แล้วมันมีไว้ทำไม?

ปกติ Struct เราจะสร้างมาเพื่อเก็บข้อมูล เช่น ชื่อ อายุ หรือราคา แต่ Unit-Like Struct ไม่มีข้อมูลให้เก็บ เราจึงใช้มันในกรณีที่เราต้องการสร้าง **Type** สำหรับเป็นตัวแทนบางอย่าง โดยไม่ต้องมีข้อมูลอยู่ข้างใน

## [Unit-Like Structs] Code ตัวอย่าง

ตัวอย่าง

```
struct Admin;  
  
fn check_admin(user: Admin)  
{  
    println!("User is Admin");  
}  
  
fn main() {  
    let admin = Admin;  
    check_admin(admin);  
}
```

ผลลัพธ์

User is Admin

Admin ไม่มีข้อมูล แต่ Rust ยังมองว่า Admin สามารถใช้เป็น Type ได้

# [Methods and the self Parameter] แนวคิด?

Method คืออะไร?

Method คือ Function ที่ถูกสร้างไว้ภายใน impl ของ Struct ใช้สำหรับกำหนดพฤติกรรมหรือการทำงานของ Struct

self คืออะไร?

self คือ Parameter พิเศษของ Method ใช้สำหรับอ้างอิงถึง Struct Instanceที่กำลังเรียก Method

ตัวอย่าง

```
struct Student {
    name: String,
    age: i32,
}

impl Student {
    fn introduce(&self) {
        println!("Hi, I'm {}. I'm {} years old.",
            self.name, self.age);
    }
}

fn main() {
    let student = Student {
        name: String::from("John"),
        age: 20,
    };

    student.introduce();
}
```

```
student
name = John
age = 20
```

ผลลัพธ์

```
Hi, I'm John. I'm 20 years old.
```

# [Associated Functions] แนวคิด?

Associated Function คือ Function ที่อยู่ภายใน  
impl ของ Struct

แต่ไม่มี self เพราะไม่ได้ทำงานกับ Struct  
Instance โดยตรง

```
struct Student {  
    name: String,  
    age: i32,  
}  
  
impl Student {  
    // Associated Function  
    fn new(name: String) -> Student {  
        Student {  
            name,  
            age: 18,  
        }  
    }  
  
    fn introduce(&self) {  
        println!("Hi, I'm {}. I'm {} years old.",  
self.name, self.age);  
    }  
}  
  
fn main() {  
    let student = Student::new(String::from("John"));  
  
    student.introduce();  
}
```

## ໄມໄວ້ Associated Function

```
struct Student {
    name: String,
    age: i32,
}

impl Student {
    fn introduce(&self) {
        println!("Hi, I'm {}. I'm {}yearsold.",
self.name, self.age);
    }
}

fn main() {
    let student = Student {
        name: String::from("John"),
        age: 20,
    };

    student.introduce();
}
```

## ໃວ້ Associated Function

```
struct Student {
    name: String,
    age: i32,
}

impl Student {
    // Associated Function
    fn new(name: String) -> Student {
        Student {
            name,
            age: 18,
        }
    }

    fn introduce(&self) {
        println!("Hi, I'm {}. I'm {} years old.",
self.name, self.age);
    }
}

fn main() {
    let student = Student::new(String::from("John"));

    student.introduce();
}
```

# Detailed Code + Live Demo

```
#[derive(Debug)]
struct UserAccount {
    username: String,
    balance: f64,
}

impl UserAccount {
    fn new(username: &str, initial_balance: f64) -> Self {
        Self {
            username: username.to_string(),
            balance: initial_balance,
        }
    }
}
```

## Demo Checklist (ทำไว้เป็น note ของ slide แทน)

☐ [ขั้นตอน Demo ที่ 1]

`#[derive(Debug)]`: ช่วยให้สามารถพิมพ์ค่าของ Struct ออกมาทางหน้าจอโดยใช้ฟอร์แมต `{:?}` เพื่อการ Debugging ได้สะดวก

☐ [ขั้นตอน Demo ที่ 2]

Struct UserAccount: การนิยามโครงสร้างข้อมูลเพื่อเก็บสแตตของผู้ใช้ ประกอบด้วย username, balance

☐ [ขั้นตอน Demo ที่ 3]

impl UserAccount: บล็อกสำหรับเขียนฟังก์ชันเฉพาะของ Struct

☐ [ขั้นตอน Demo ที่ 4]

fn new(...): เป็น Associated Function (ไม่ได้รับ self) ทำหน้าที่เป็น Constructor เพื่อสร้าง Instance ใหม่ของ Struct

# Detailed Code + Live Demo

```
fn get_balance(&self) -> f64 {
    self.balance
}

fn deposit(&mut self, amount: f64) {
    if amount > 0.0 {
        self.balance += amount;
        println!("ฝากเงินสำเร็จ: +${:.2} | ยอดคงเหลือปัจจุบัน: ${:.2}", amount, self.balance);
    } else {
        println!("จำนวนเงินฝากต้องมากกว่า 0");
    }
}
```

## Demo Checklist (ทำไว้เป็น note ของ slide แทน)

☐ [ขั้นตอน Demo ที่ 5]

fn get\_balance(&self): เป็น Immutable Method ยืมอ่านค่าฟิลด์ balance โดยไม่มีการเปลี่ยนแปลงข้อมูลภายใน

☐ [ขั้นตอน Demo ที่ 6]

fn deposit(&mut self, ...) & fn withdraw(&mut self, ...): เป็น Mutable Methods ยืมสิทธิ์เข้าถึงเพื่อปรับแต่งแก้ไขค่า balance ภายใน Struct

# Detailed Code + Live Demo

```
fn withdraw(&mut self, amount: f64) -> Result<f64, String> {
    if amount <= 0.0 {
        Err("จำนวนเงินถอนต้องมากกว่า 0".to_string())
    } else if amount > self.balance {
        Err("ยอดเงินคงเหลือไม่เพียงพอ".to_string())
    } else {
        self.balance -= amount;
        Ok(self.balance)
    }
}

fn close_account(self) -> f64 {
    println!("ปิดบัญชีของ {} สำเร็จ คืนเงินคงเหลือ: ${:.2}", self.username, self.balance);
    self.balance
}
}
```

## Demo Checklist (ทำไว้เป็น note ของ slide แทน)

□ [ขั้นตอน Demo ที่ 6]

fn deposit(&mut self, ...) & fn withdraw(&mut self, ...): เป็น Mutable Methods ยืมสิทธิ์เข้าถึงเพื่อปรับแต่งแก้ไขค่า balance ภายใน Struct

□ [ขั้นตอน Demo ที่ 7]

fn close\_account(self): เป็น Ownership Consumer Method โดยรับ self ไปตรง ๆ ทำให้ Instance นั้นถูกย้าย Ownership (Move) และย้อนกลับมาใช้งานอีกไม่ได้หลังจบคำสั่ง เพื่อความปลอดภัยด้านหน่วยความจำ

# Detailed Code + Live Demo

```
fn main() {
    let mut my_account = UserAccount::new("Alice", 100.0);
    println!("เริ่มต้นบัญชี: {:?}", my_account);

    println!("ยอดเงินเริ่มต้น: ${:.2}", my_account.get_balance());

    my_account.deposit(50.0);

    match my_account.withdraw(30.0) {
        Ok(new_balance) => println!("ถอนเงินสำเร็จ ยอดคงเหลือ: ${:.2}", new_balance),
        Err(e) => println!("เกิดข้อผิดพลาด: {}", e),
    }

    let refunded = my_account.close_account();
    println!("เงินคืนเข้ามือ: ${:.2}", refunded);
}
```

## Demo Checklist (ทำไว้เป็น note ของ slide แทน)

- ☐ [ขั้นตอน Demo ที่ 1]  
สร้าง Instance ของ UserAccount
- ☐ [ขั้นตอน Demo ที่ 2]  
ปริ้นท์ข้อมูลดิบของ Struct ออกมาดู (Debug Mode)
- ☐ [ขั้นตอน Demo ที่ 3]  
ดึงยอดเงินออกมาแสดงผล
- ☐ [ขั้นตอน Demo ที่ 4]  
ฝากเงินเพิ่ม 50.0
- ☐ [ขั้นตอน Demo ที่ 5]  
ถอนเงิน 30.0 พร้อมตรวจสอบข้อผิดพลาดด้วย match
- ☐ [ขั้นตอน Demo ที่ 6]  
ปิดบัญชี คืนเงินคงเหลือทั้งหมด และทำลาย Instance

# Rust vs [Other Language]

| Aspect               | Rust                                                                             | C++                                                                   | Java                                                                             | Python                                                                  |
|----------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Syntax               | แยกนิยาม Data (struct) ออกจาก Method/Function (impl)                             | รวม Data และ Method ไว้ใน struct หรือ class                           | รวม Data และ Method ใน class                                                     | รวม Data และ Method ใน class                                            |
| Semantics / Behavior | ส่ง Receiver ชัดเจน (&self, &mut self, self)                                     | ใช้ this เป็น pointer                                                 | ใช้ this เป็น reference                                                          | ใช้ <b>self</b> เป็นพารามิเตอร์ตัวแรก เพื่ออ้างถึง Object ปัจจุบัน      |
| Type System          | Statically & Strongly typed/Trait                                                | Static Typing + Multiple Inheritance + Virtual Function               | Statically & Strongly typed Single Inheritance + Interface                       | Dynamically & Strongly typed/Duck Typing                                |
| Memory Management    | Ownership + Scope เมื่อไม่ใช้งานแล้ว ระบบจะคืนหน่วยความจำอัตโนมัติ ไม่ต้องใช้ GC | Stack / Heap จัดการอายุข้อมูลด้วย RAII, new/delete หรือ Smart Pointer | Object โดยทั่วไปอยู่ Heap จัดการหน่วยความจำด้วย Garbage Collector (GC) อัตโนมัติ | Object โดยทั่วไปอยู่ Heap ใช้ Reference Counting เป็นหลัก และ Cyclic GC |
| Safety               | Compile-time Safety Ownership / Borrowing                                        | ยัดเยียดสูง / Dangling Pointer / Undefined Behavior                   | GC ช่วยด้าน Memory Safety แต่ยังมีโอกาสเกิด NullPointerException ตอน Runtime     | จัดการ Memory อัตโนมัติ/ TypeError / AttributeError / IndexError        |

Analysis: Rust เลือกแนวทางการออกแบบ "Separation of Concerns" และ "Composition over Inheritance" โดยแยกนิยามโครงสร้างข้อมูล (struct) ออกจากพฤติกรรมการทำงาน (impl) อย่างเด็ดขาด เพื่อตัดปัญหาความซับซ้อนของ Type Hierarchy และ Fragile Base Class ในภาษาเชิงวัตถุดั้งเดิมอย่าง C++ หรือ Java

# PPL Perspective — มุมมองเชิง Programming Language

## Syntax

- ใช้ struct สำหรับข้อมูล และ impl สำหรับ Method
- ใช้ let, fn, match และไม่ใช่ class

## Semantics

- struct ใช้เก็บข้อมูล ส่วน impl ใช้กำหนดพฤติกรรมของ Type
- Ownership และ Borrowing กำหนดว่าใครมีสิทธิ์ใช้ข้อมูล

## Type System

- เป็น Static + Strongly Typed
- ใช้ Trait กำหนดพฤติกรรมร่วม แทนการสืบทอดแบบ Class

## Memory / Resource

- ใช้ Ownership + Borrowing + Scope จัดการหน่วยความจำ
- คืนหน่วยความจำอัตโนมัติเมื่อเจ้าของออกจาก Scope โดยไม่ใช่ GC

## Abstraction

- ใช้ Trait เพื่อสร้าง Abstraction และ Module เพื่อจัดขอบเขตของโค้ด
- ควบคุมการเข้าถึงข้อมูลด้วย pub และระบบ Ownership

## Why Rust?

- เพิ่ม Memory Safety + Reliability โดยตรวจสอบปัญหาสำคัญตั้งแต่ Compile-time
- พร้อมรักษา Performance ใกล้เคียงภาษาระดับระบบ

# Key Takeaways — สรุปสิ่งที่ควรจำ

- 1 **Struct = Data:** struct ใช้รวมข้อมูลที่เกี่ยวข้องกันไว้ใน Type เดียว
- 2 **impl = Behavior:** impl ใช้กำหนด Methods และพฤติกรรมของ struct
- 3 **self กำหนดการเข้าถึงข้อมูล:** &self = อ่านข้อมูล, &mut self = แก้ไขข้อมูล, self = รับ Ownership
- 4 **Rust กับ PPL:** Rust ใช้ **Ownership + Borrowing** ควบคุมการเข้าถึง Resource และตรวจสอบความปลอดภัยของ Memory ตั้งแต่ Compile-time ซึ่งเป็นแนวคิดสำคัญด้าน **Memory Management** และ **Safety** ของภาษา

# References — เอกสารอ้างอิง

- The Rust Programming Language (The Rust Book) — <https://doc.rust-lang.org/book/>
- Rust by Example — <https://doc.rust-lang.org/rust-by-example/>
- [<https://www.w3schools.com/rust>]
- [<https://users.rust-lang.org/>]

*GitHub Repository:* [[rust-tutorial-2569/08-structs-methods at main · soonklang/rust-tutorial-2569](https://github.com/soonklang/rust-tutorial-2569)]

# Questions & Answers

กลุ่ม [8] — [Structs & Methods]